

Prisma Fundamentals

A beginner-friendly guide to using Prisma ORM with MySQL and JavaScript in Visual Studio Code.

1. What is Prisma?

Prisma is an ORM (Object Relational Mapping). It allows JavaScript or TypeScript applications to work with database records using programming-language methods instead of writing SQL for every operation. Prisma generates a Prisma Client that communicates with the database.

2. Why Use Prisma?

- Easier database queries from JavaScript.
- Useful autocomplete in VS Code.
- Database models are defined in one schema file.
- Migrations help manage database structure.
- Works well with Node.js applications.

3. Install Node.js

Install Node.js on Windows. Then open the VS Code terminal and check the installation:

```
node -v
npm -v
```

4. Create a New Project

```
mkdir prisma-demo
cd prisma-demo
npm init -y
```

5. Install Prisma

```
npm install -D prisma
npm install @prisma/client
```

6. Initialize Prisma

```
npx prisma init
```

This creates a prisma folder containing schema.prisma and an .env file.

7. Configure MySQL

In .env, set the connection string for your local MySQL server. For a typical XAMPP setup, the username is root and the password may be blank depending on your configuration.

```
DATABASE_URL="mysql://root:YOUR_PASSWORD@localhost:3306/school"
```

Replace YOUR_PASSWORD with your actual MySQL password. Make sure the school database exists.

8. schema.prisma

The Prisma schema describes the database connection and the models used by your application.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

model Student {
  id    Int    @id @default(autoincrement())
  name  String
  age   Int
}
```

9. Understanding the Model

id is an integer primary key. @id marks the primary key. @default(autoincrement()) lets the database generate IDs automatically. name is a String and age is an Int.

10. Migration

```
npx prisma migrate dev --name init
```

A migration applies the schema to the MySQL database and records the database change.

11. Prisma Client

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
```

Prisma Client is the JavaScript API generated from your Prisma schema.

12. Create

```
const student = await prisma.student.create({
  data: {
    name: "John",
    age: 15
  }
});
```

13. Read

```
const students = await prisma.student.findMany();

const student = await prisma.student.findUnique({
  where: { id: 1 }
});
```

14. Update

```
await prisma.student.update({
  where: { id: 1 },
  data: { age: 16 }
});
```

15. Delete

```
await prisma.student.delete({
  where: { id: 1 }
});
```

16. Useful Prisma Methods

findMany() gets multiple records.
findUnique() gets one unique record.
findFirst() gets the first matching record.
create() creates one record.
createMany() creates multiple records.
update() changes one record.
delete() removes one record.
count() counts records.

17. Filtering

```
const students = await prisma.student.findMany({
  where: {
    age: { gte: 16 }
  }
});
```

This finds students whose age is at least 16.

18. Sorting

```
const students = await prisma.student.findMany({
  orderBy: {
    age: "desc"
  }
});
```

19. Prisma Studio

```
npx prisma studio
```

Prisma Studio opens a browser interface where you can view and edit database records. It is useful for checking your data while learning.

20. Project Structure

```
prisma-demo/
├── prisma/
│   └── schema.prisma
```

```
node_modules/  
.env  
package.json  
index.js
```

21. Typical Prisma Workflow

1. Create or update schema.prisma.
2. Check the DATABASE_URL in .env.
3. Run a migration.
4. Use Prisma Client in JavaScript.
5. Test the database operations.
6. Use Prisma Studio to inspect the data.

22. Mini Project – Student Management System

Build a Student Management System using JavaScript, MySQL and Prisma.

Requirements:

- Add a student.
- Display all students.
- Search students.
- Update student information.
- Delete students.
- Store all data in MySQL through Prisma.

```
model Student {  
  id    Int    @id @default(autoincrement())  
  name  String  
  age   Int  
  email String  
}
```

23. Prisma Does Not Replace MySQL

MySQL is still the database that stores the actual data. Prisma is the tool that makes it easier for JavaScript code to communicate with MySQL. Understanding basic SQL remains useful when using Prisma.

24. Summary

The main Prisma concepts are the Prisma schema, models, migrations, Prisma Client, CRUD operations, filtering, sorting, and Prisma Studio.